Database – No SQL – Mongodb

Session 6 –

1. **Given a MongoDB collection 'orders' with documents containing fields: userId, items (array of {product, quantity, price}), and orderDate, use the $match stage to find all orders placed in the last 30 days.**

Ans:

Assuming orderDate is stored as a MongoDB Date:

```
db.orders.aggregate([
    {
        $match: {
            orderDate: {
                $gte: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000)
            }
        }
    }
])
```

2. **Using the 'orders' collection, write an aggregation pipeline that groups orders by userId and calculates the total amount spent by each user using the $group stage.

Hint: Use $sum to total the value of all items in each user's orders.**

Ans:

```
db.orders.aggregate([
    {
        $unwind: "$items"
    },
    {
        $group: {
            _id: "$userId",
            totalSpent: {
                $sum: {
                    $multiply: [
                        "$items.quantity",
                        "$items.price"
                    ]
                }
            }
        }
    }
])
```

Example output:

```
{
    _id: "U101",
    totalSpent: 2450
}
{
    _id: "U102",
    totalSpent: 1875
}
```

3. **Create an aggregation pipeline on a 'restaurants' collection (fields: name, cuisine, city, rating) to find the top 3 highest-rated cafes in Ahmedabad. Use $match for cuisine, $sort for rating, and $limit for top results.**

Ans:

```
db.restaurants.aggregate([
    {
        $match: {
            cuisine: "Cafe",
            city: "Ahmedabad"
        }
    },
    {
        $sort: {
            rating: -1
        }
    },
    {
        $limit: 3
    }
])
```

4. **In a 'movies' collection (fields: title, genre, boxOffice), use $group to calculate the total box office collection for each genre and then use $project to display only genre and totalCollection fields in the output.

Constraint: Do not include the default _id field in your final projection.**

Ans:

```
db.movies.aggregate([
    {
        $group: {
            _id: "$genre",
            totalCollection: {
                $sum: "$boxOffice"
```

```
            }
        }
    },
    {
        $project: {
            _id: 0,
            genre: "$_id",
            totalCollection: 1
        }
    }
])
```

Example:

```
{
    genre: "Action",
    totalCollection: 125000000
}
{
    genre: "Comedy",
    totalCollection: 85000000
}
```

5. **Use ChatGPT or GitHub Copilot to generate an aggregation pipeline that produces a daily sales report from a 'payments' collection (fields: userId, amount, paymentDate), showing total sales per day for the last week. Paste the generated code and briefly describe any changes you made to fit your collection's structure.**

Ans:

A suitable generated aggregation pipeline for the payments collection is:

```
db.payments.aggregate([
    {
        $match: {
            paymentDate: {
                $gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000)
            }
        }
    },
    {
        $group: {
            _id: {
                $dateToString: {
                    format: "%Y-%m-%d",
                    date: "$paymentDate"
                }
            },
```

```
            totalSales: {
                $sum: "$amount"
            }
        }
    },
    {
        $project: {
            _id: 0,
            date: "$_id",
            totalSales: 1
        }
    },
    {
        $sort: {
            date: 1
        }
    }
])
```